



University
of Exeter

COMM117

Large Language Models and Application

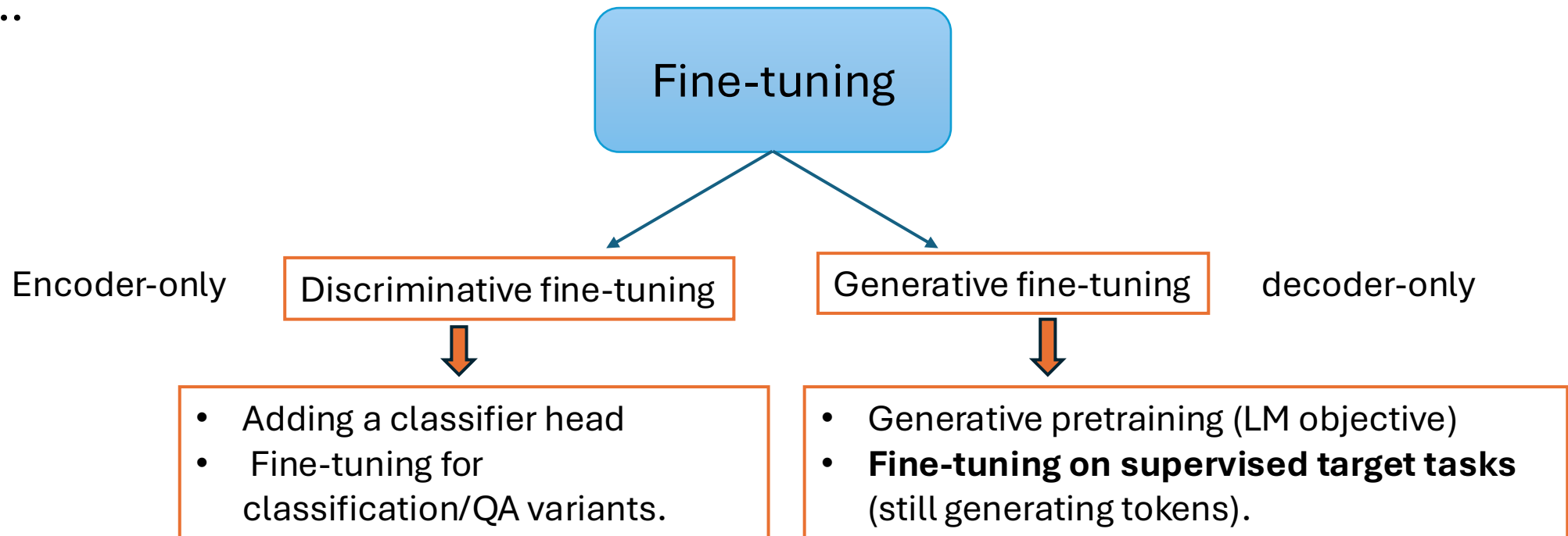
Base-model stability → alignment stability

- ✓ Why training Transformers is fragile at scale
 - Post-training is built on top of a base model. → **stable base training is a prerequisite** for predictable post-training.

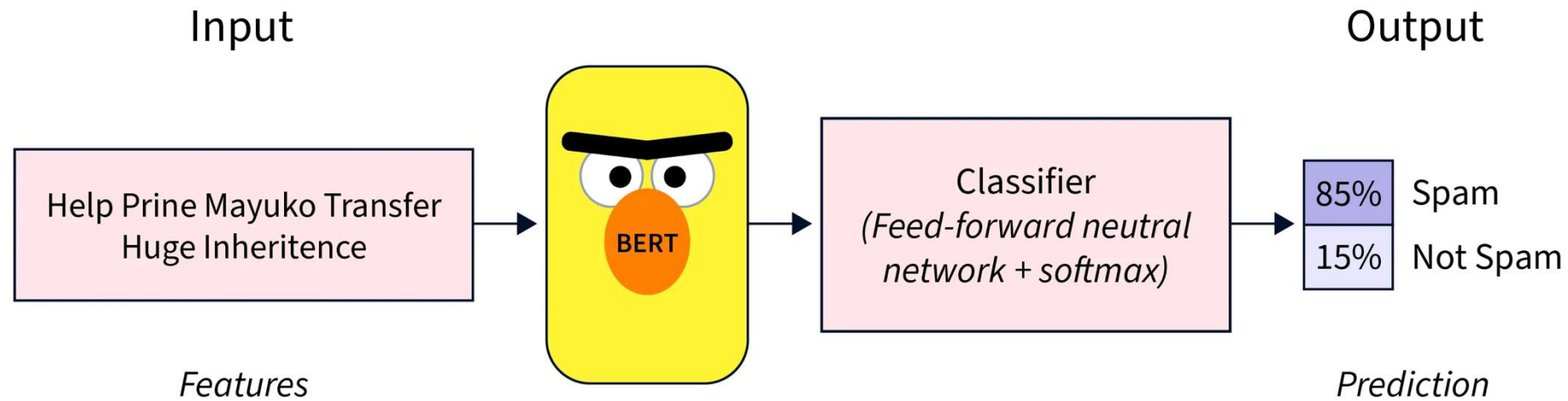


Fine-tuning

- Task finetune,
- Domain adaptation (continued pretraining),
- Multi-task finetune
- Supervised Finetuning
- ...



Fine-tuning- BERT



During fine-tuning, only the **weights of the additional layer** added to the pre-trained BERT model are updated.

$$\hat{y} = \text{softmax}(W h_{CLS} + b)$$

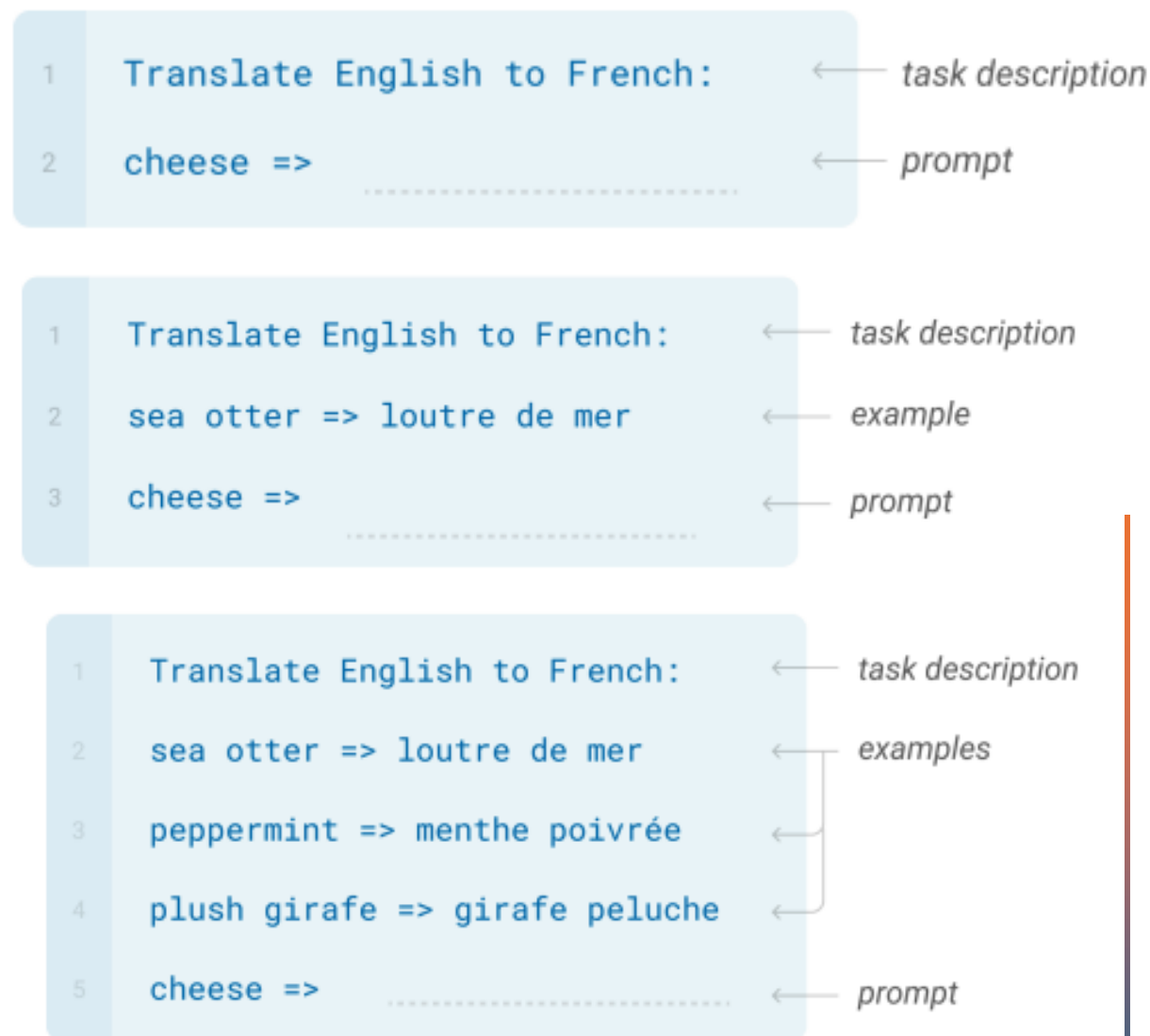
↓
Target class

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



In-Context Learning



Fine-tuning- Classic task fine-tuning (GPT)

Table 1: A list of the different tasks and datasets used in our experiments.

Task	Datasets
Natural language inference	SNLI [5], MultiNLI [66], Question NLI [64], RTE [4], SciTail [25]
Question Answering	RACE [30], Story Cloze [40]
Sentence similarity	MSR Paraphrase Corpus [14], Quora Question Pairs [9], STS Benchmark [6]
Classification	Stanford Sentiment Treebank-2 [54], CoLA [65]

Model: decoder-only (autoregressive generator)

Output: text tokens (a completion)

Objective: next-token prediction on **task-formatted text**

$\max \log p_{\theta} (\text{output tokens} \mid \text{prompt tokens})$

Example: summarization $\rightarrow$ prompt = article + “TL;DR:” $\rightarrow$ output = summary text.

https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

Supervised Fine-Tuning (SFT)-Data

➤ Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
➤ Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

s style

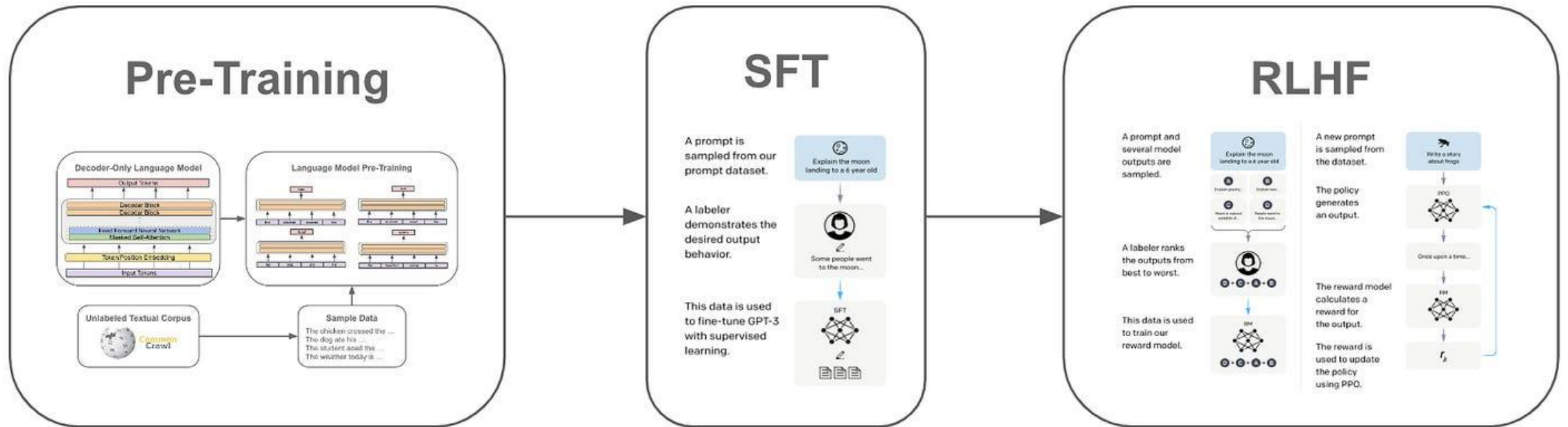
Table 5: SFT annotation — example of a *helpfulness* (top) and *safety* (bottom) annotation for SFT, where the annotator has written both the prompt and its answer.

Introduction Today

1. PEFT/LoRA (efficiency + modularity)
2. Preference optimization: RLHF vs DPO
(align to human preferences)
3. InstructGPT

Base model vs assistant model (the behaviour gap)

Alignment



Limitations of SFT

Scale Up the model

- **Computational Cost:**

Fine-tuning requires significant GPU/TPU resources.

- **Overfitting Risks:**

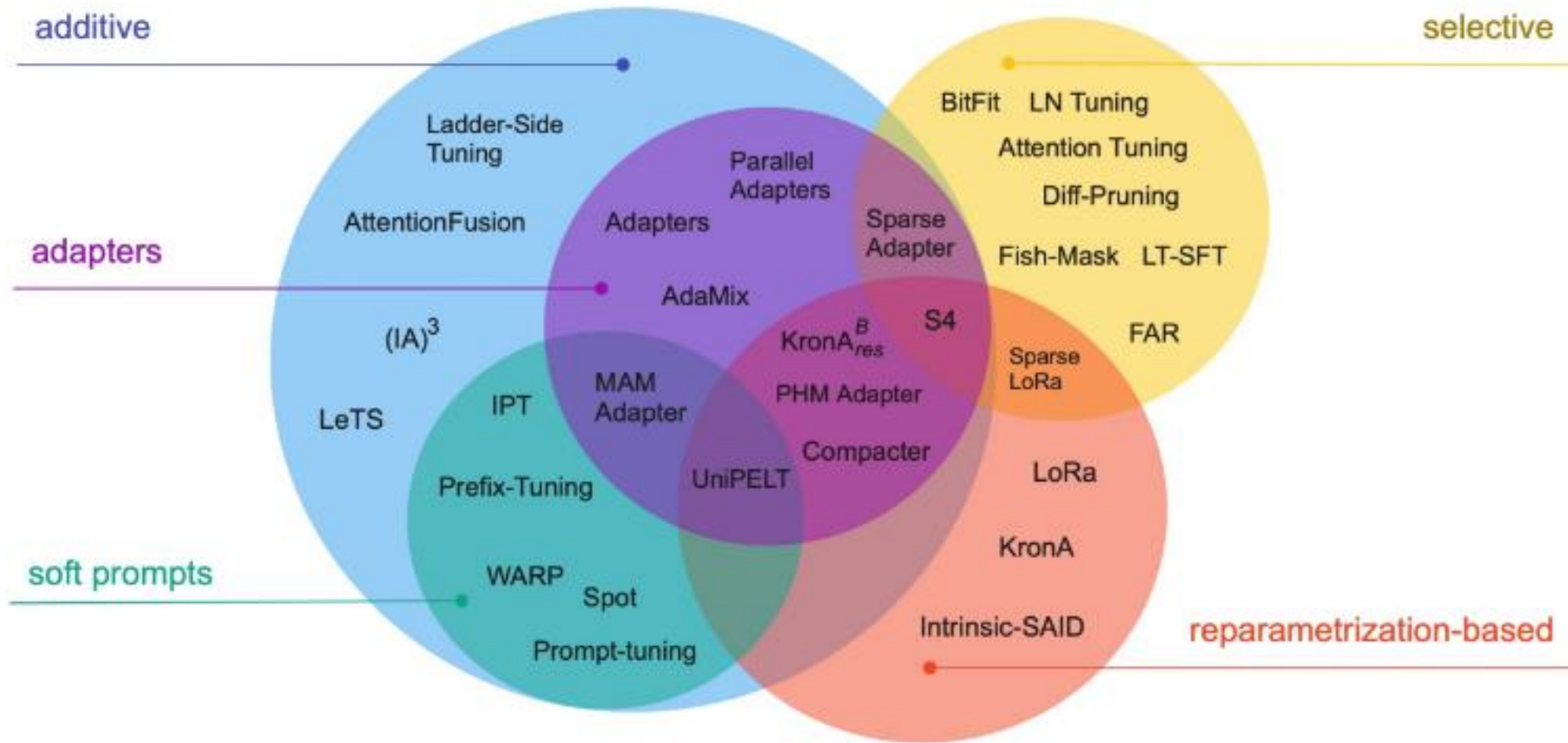
Retraining all parameters can **overfit** on small datasets.

- **Deployment:**

Large fine-tuned models can be tricky to deploy efficiently.

When you fine-tune a model, you're essentially retraining all its parameters to adapt to your specific dataset.

Parameter Efficient Fine Tuning (PEFT)



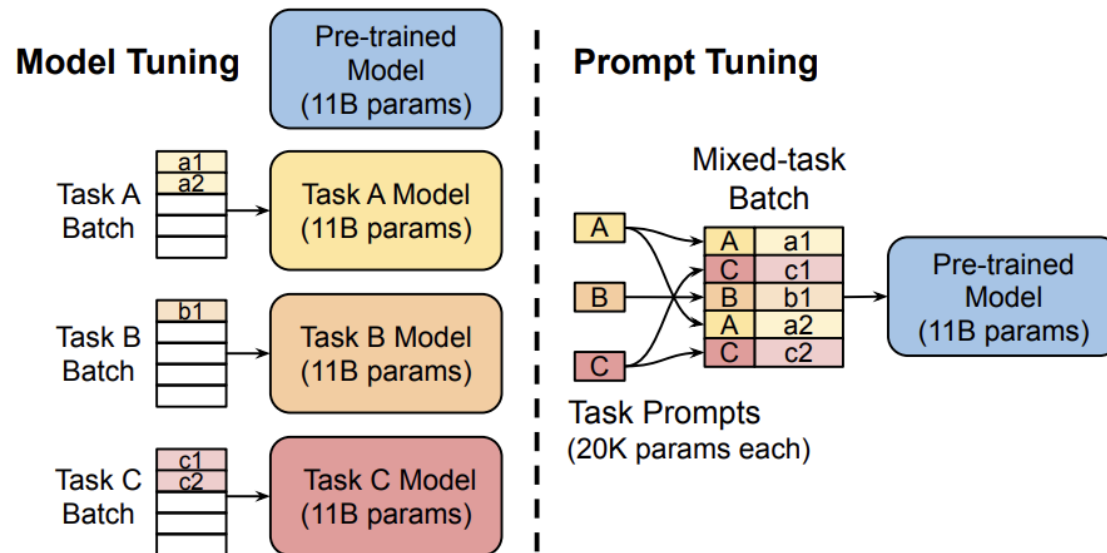
- IA^3

Parameter-Efficient Fine-Tuning (PEFT)

- **Resource-Constrained Environments:** Training with **fewer parameters** means **lower GPU memory** usage and faster iteration cycles. Perfect when you're working with limited compute.
- **Small Datasets:** PEFT **minimizes the risk of overfitting** since it doesn't modify the entire model.
- **Production Deployment:** Models trained with PEFT often have smaller memory footprints, making them **easier to deploy**, especially in edge devices.

Prompt Tuning

Soft Prompts: Fine-tuning applied directly to the model's **input embeddings**, enabling task-specific adaptation without modifying the model's internal parameters.



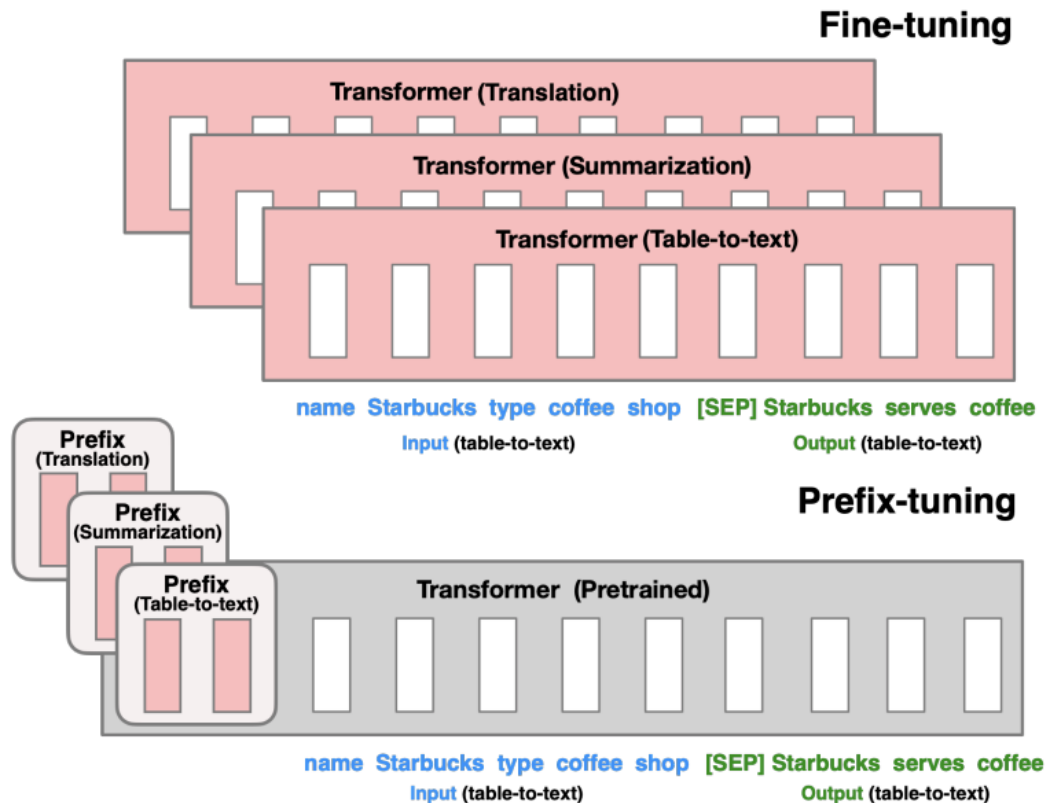
What you train: a small set of **learned “virtual tokens”** (embeddings) prepended to the input.

- learn an **embedding matrix** for prompt tokens
- At inference, you **prepend these learned tokens to every input** (often per task).

The Power of Scale for Parameter-Efficient Prompt Tuning

Prefix tuning

Instead of adding the prompt tensor to only the input layer, prefix tuning **adds trainable parameters** are **prepended to the hidden states of all layers**.



What you train: learned vectors that act like a **prefix in every transformer layer**, typically injected into **attention key/value** states.

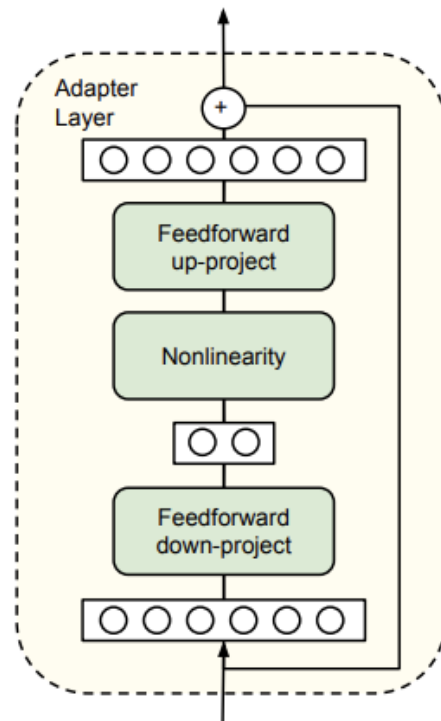
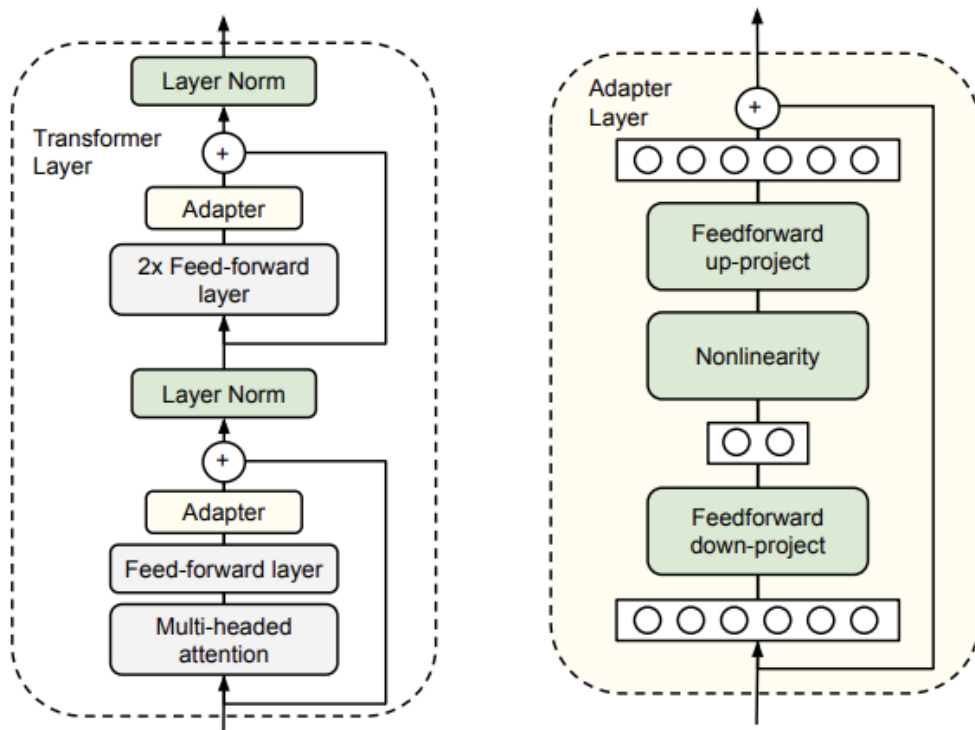
At inference, for each transformer layer, you **prepend a set of learned key/value vectors** to the layer's attention KV.

The base model weights stay frozen; **you only load the prefix parameters.**

[Prefix-Tuning: Optimizing Continuous Prompts for Generation](#)

Adapter

- **Adapters:** Small **bottleneck MLP modules** inserted into each transformer block, allowing adaptation to new tasks.



A common adapter is:

$$h \leftarrow h + W_{up} \sigma(W_{down} h)$$

where $W_{down} \in \mathbb{R}^{d \times r}$ and $W_{up} \in \mathbb{R}^{r \times d}$ with $r \ll d$.

Pros

- Strong performance across many tasks (historically very effective in NLP)

Cons

- Adds extra layers $\rightarrow$ some inference overhead
- More parameters than prompt/prefix

Overview

- **Prompt tuning:** only **input embeddings** change (learned tokens).
- **Prefix tuning:** **attention KV** gets extra learned content at **each layer**.
- **Adapters:** add small **trainable sub-networks** inside **each block**.

Want *tiny* task-specific footprint and quick experiments



Overview

- **Prompt tuning:** only **input embeddings** change (learned tokens).
- **Prefix tuning:** **attention KV** gets extra learned content at **each layer**.
- **Adapters:** add small **trainable sub-networks** inside **each block**.

Need robust adaptation across layers with modular components



Overview

- **Prompt tuning:** only **input embeddings** change (learned tokens).
- **Prefix tuning:** **attention KV** gets extra learned content at **each layer**.
- **Adapters:** add small **trainable sub-networks** inside **each block**.

- ✓ Want *tiny* task-specific footprint and quick experiments → **prompt tuning**
- ✓ Need more control but still very small params → **prefix tuning**
- ✓ Need robust adaptation across layers with modular components → **adapters**

In modern LLMs?

LoRA (Low-Rank Adaptation) **simple + strong!**

LoRA core idea

LoRA (Low-Rank Adaptation): LoRA modifies **low-rank matrices**, allowing for efficient parameter updates.

- Replace weight update: $\mathbf{W} \leftarrow \mathbf{W} + \Delta\mathbf{W}$
- **Constrain $\Delta\mathbf{W}$** to low rank: $\Delta\mathbf{W} = \mathbf{B} \mathbf{A}$ with rank r

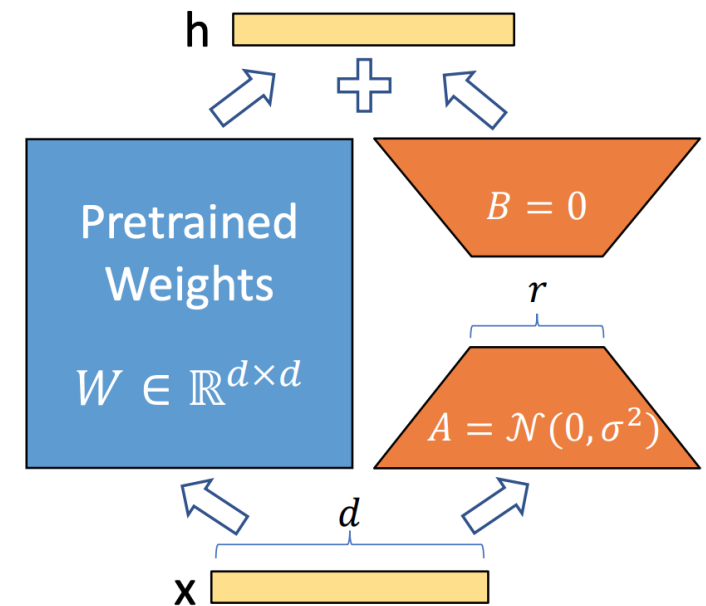


Figure 1: Our reparametrization. We only train A and B .

LoRA

- $M \times N$, with $M=100$ and $N=100$.
- LoRA “breaks” the $M \times N$ matrix down $\rightarrow$ instead of updating 100×100 weights, only a small fraction of 10,000 parameters are involved

Reconstructing delta W from A and B

The diagram illustrates the reconstruction of the weight matrix ΔW from two smaller matrices, A and B . Matrix A is represented by a purple rectangle with dimensions $M \times r$. Matrix B is represented by an orange rectangle with dimensions $r \times N$. The product of A and B is shown as a blue rectangle with dimensions $M \times N$. The dimensions are indicated by curly braces and labels: M for rows and N for columns. The resulting matrix is labeled "same dimensions as the original weight matrix W".

$$\text{LoRA}(A, B) = A \times B = \Delta W$$

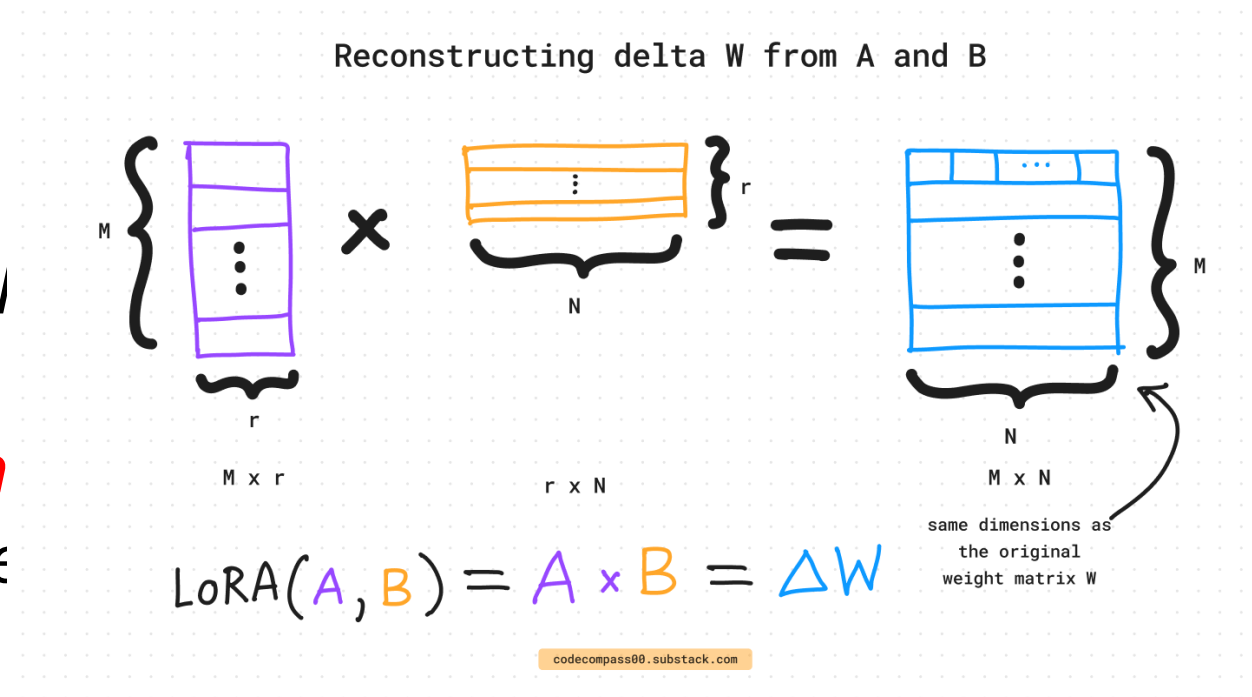
LoRA

- For a linear layer: $y = Wx$
- LoRA: $y = (W + \alpha/r \cdot BA)x$

r controls how much new behaviour the adapter can store

*α controls the **magnitude of the learn update** relative to the frozen base. (step size)*

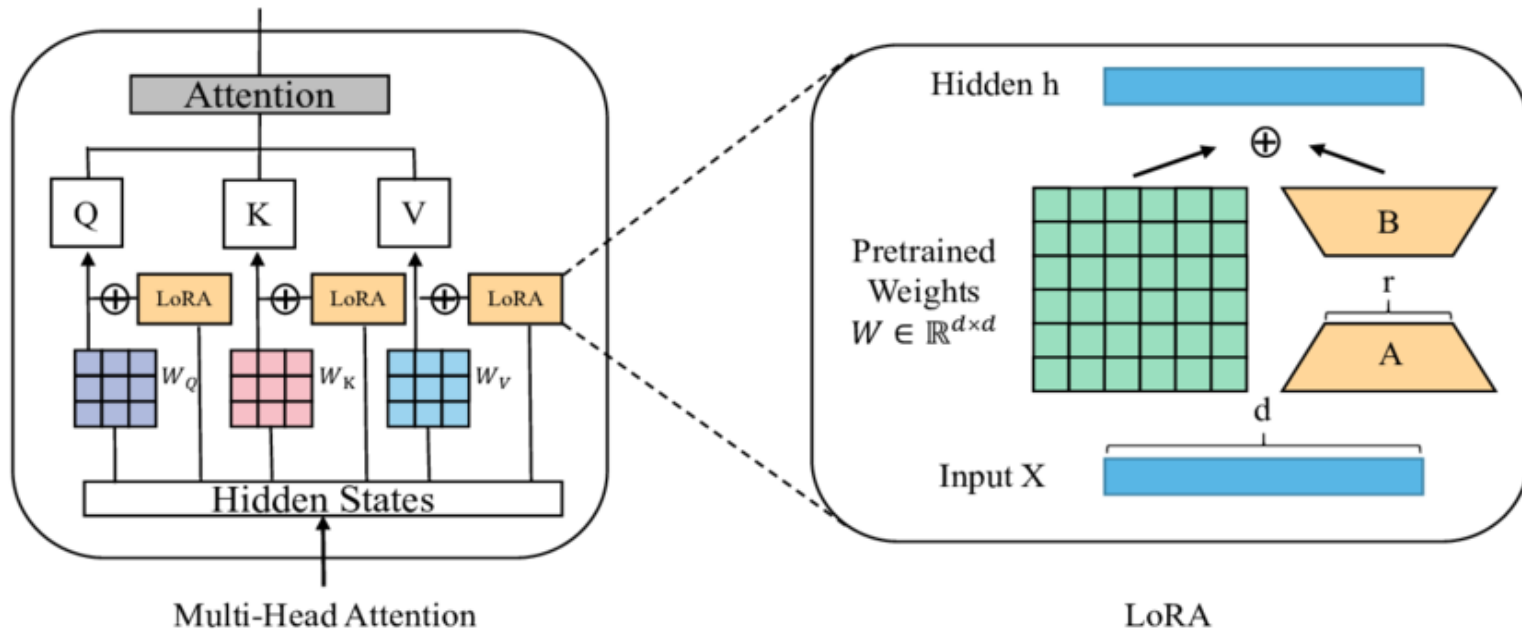
- Train A,B; keep W frozen



LoRA

- Where to insert LoRA in a transformer

→ attention projections $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$; sometimes MLP $\mathbf{W}_1/\mathbf{W}_2$

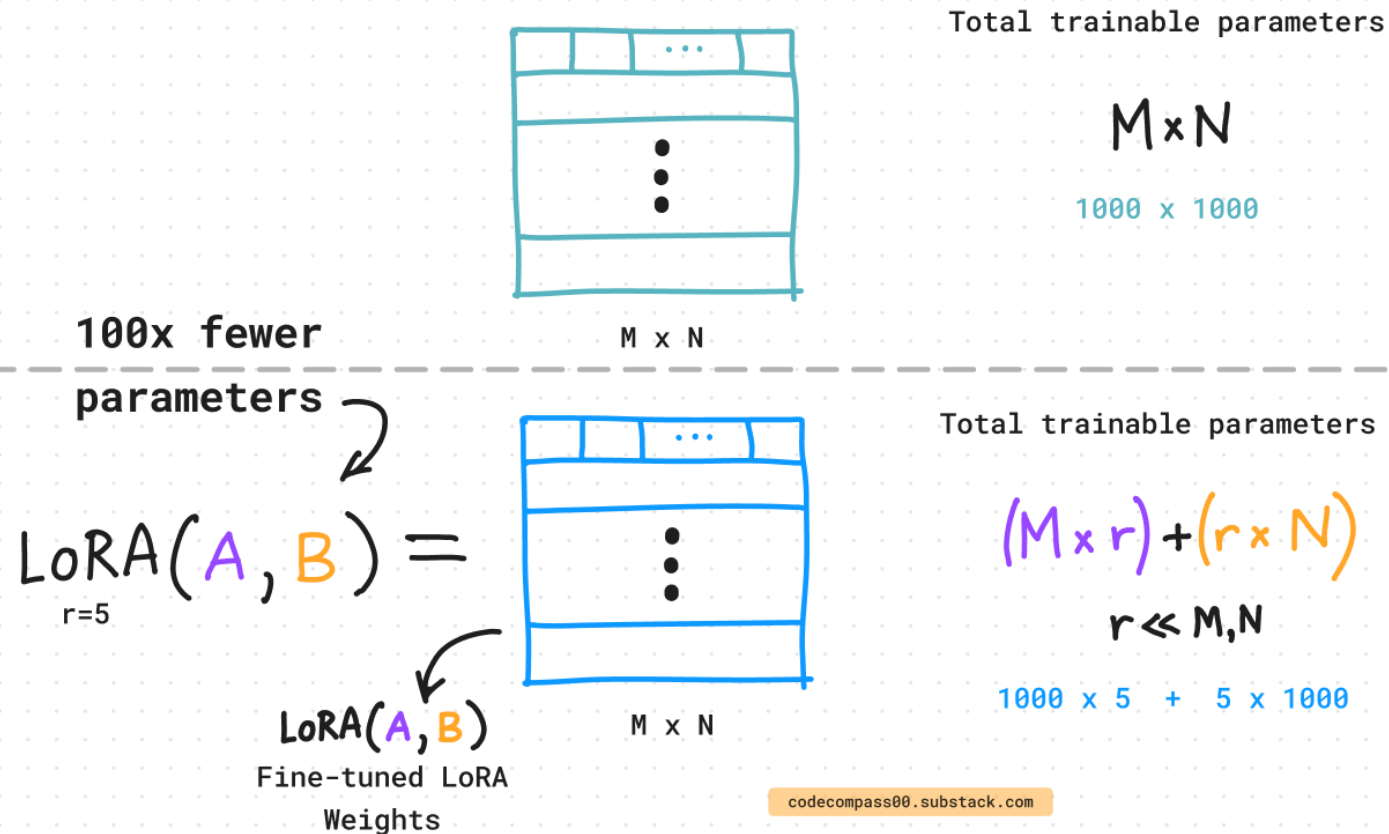


Trade-offs: more modules $\rightarrow$ capacity $\uparrow$ but memory/latency $\uparrow$

In practical: start with Q,V (or Q,V,O) then expand if needed

LoRA Parameter counting

Fintune < 1% of total parameters (10,000/1,000,000 = 1%)



Full FT: $d \times d$ trainable
LoRA: $2 \cdot d \cdot r$ trainable
 $r=8$ or 16 is often enough for strong gains.

LoRA

During Training

- Initialization: Initialize matrices A and B with **random values**.
- Forward Pass: **compute** the low-rank approximation as **$A \times B$** and add this to the original weights.
- Backward Pass: **Only update A and B during backpropagation**.

During Inference

Inference: the product $A \times B$ is precomputed and added to the original weights

LoRA

During Training

- Initialization: Initialize matrices A and B with **random values**.
- Forward Pass: **compute** the low-rank approximation as $A \times B$ and add this to the original weights.
- Backward Pass: **Only update A and B during backpropagation**.

During Inference

Inference: the product $A \times B$ is precomputed and added to the original weights

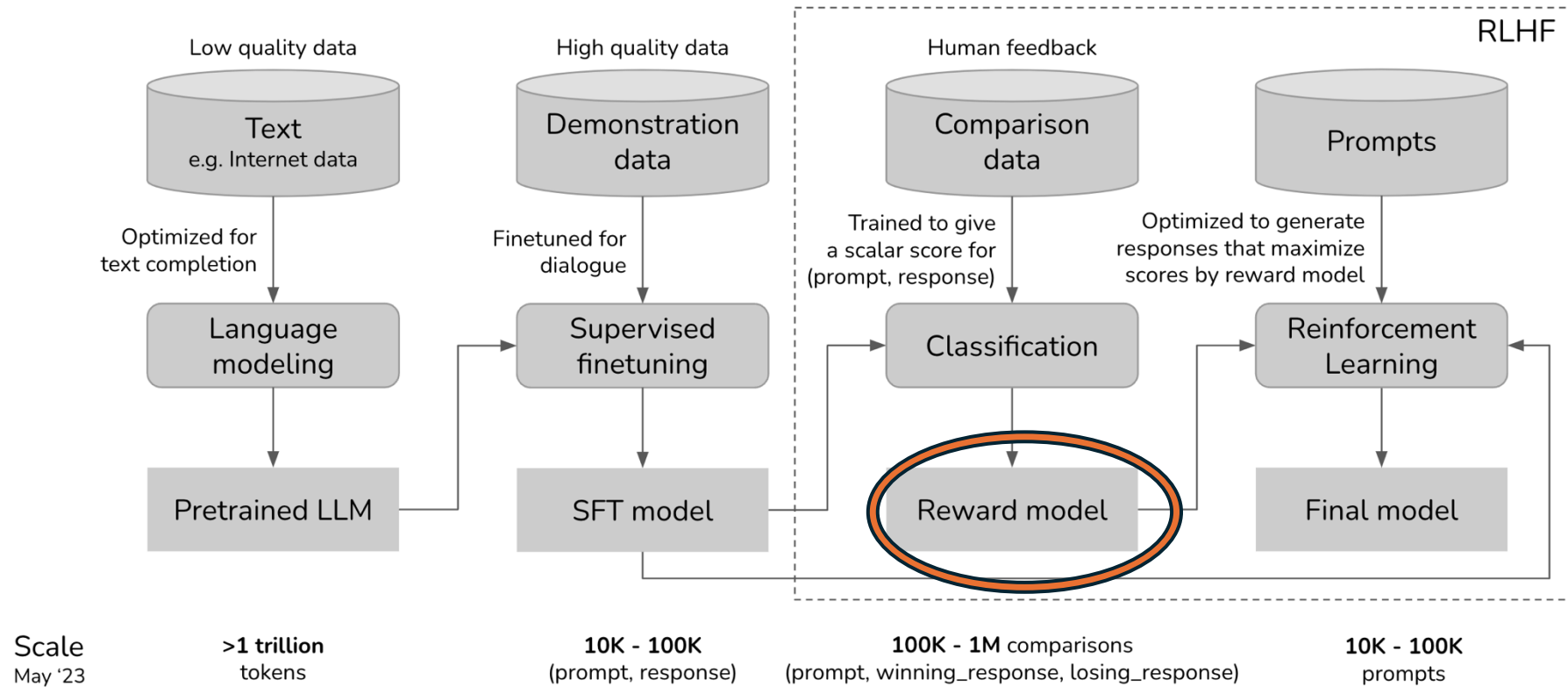
Practical tips / failure modes

- Data quality dominates
- Rank too low $\rightarrow$ underfit; too high $\rightarrow$ overfit + slower
- LoRA on wrong modules $\rightarrow$ little gain

Preference optimization: RLHF → DPO

- SFT optimizes likelihood of *one* answer
- But we actually care about a *ranking* (helpfulness/safety preferences)

RLHF(Reinforcement Learning from Human feedback)



1. Collect comparisons (A vs B)
2. Train Reward Model (RM)
3. Optimize policy with RL

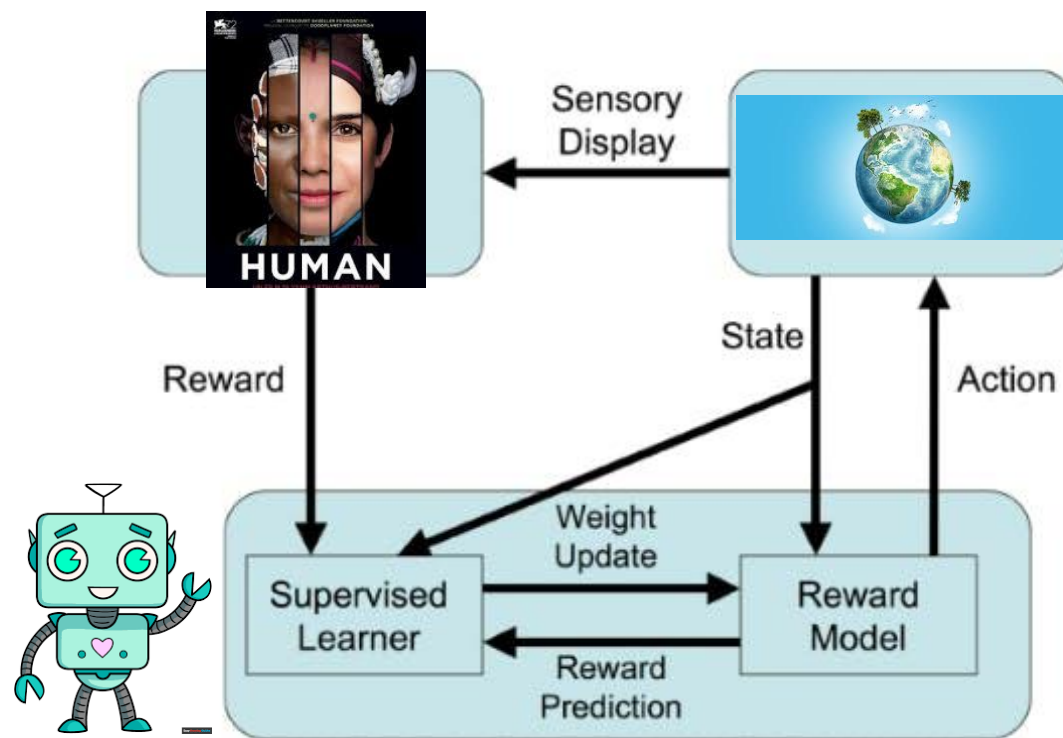
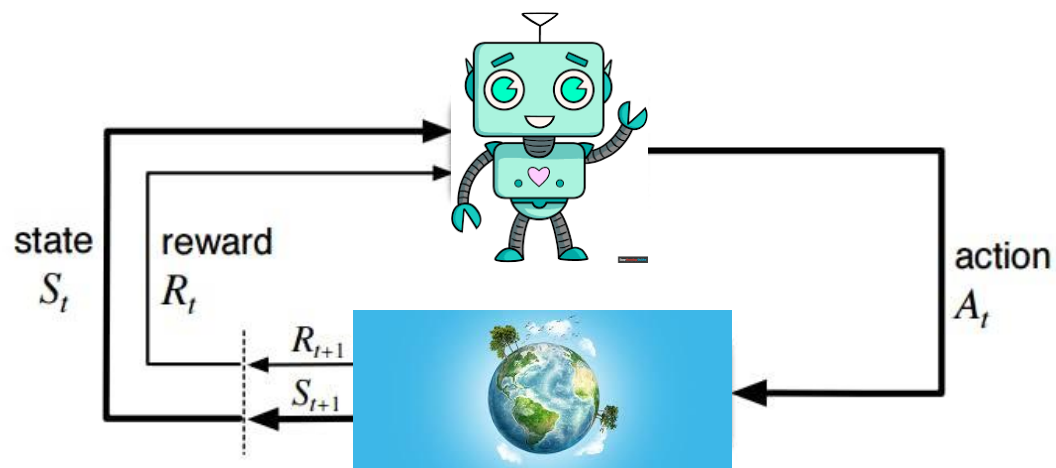
Examples
Scale
May '23

GPT-x, Gopher, **Falcon**,
LLaMa, **Pythia**, **Bloom**,
StableLM

Dolly-v2, Falcon-Instruct

InstructGPT, ChatGPT,
Claude, **StableVicuna**

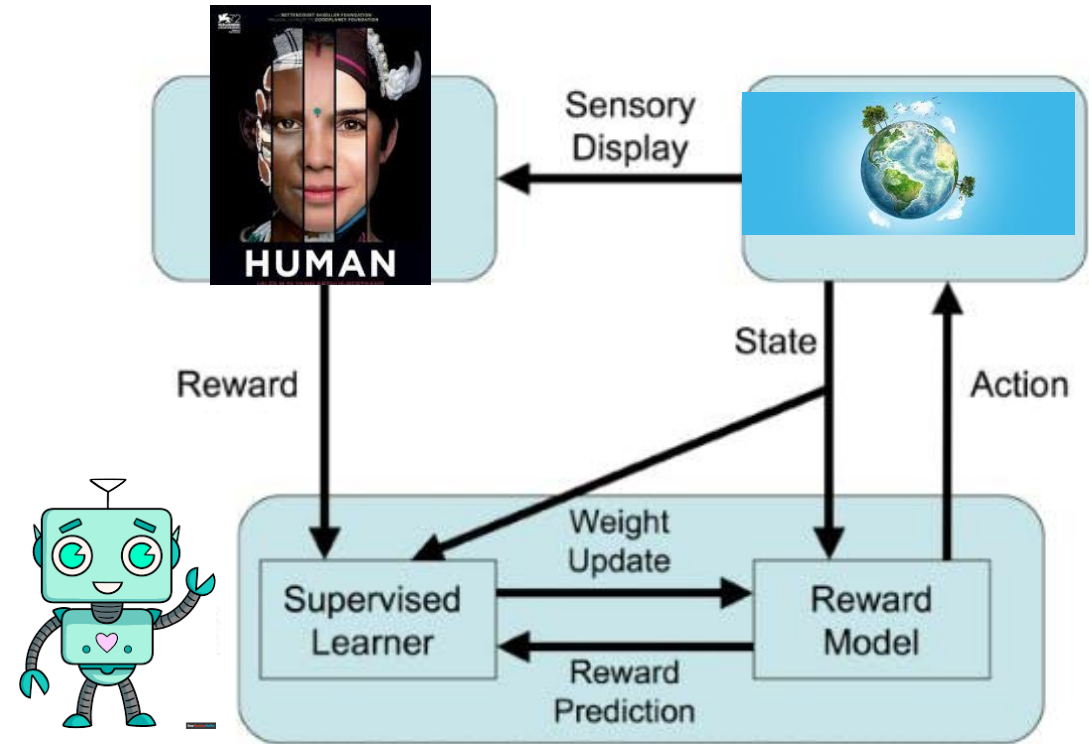
RLHF(Reinforcement Learning from Human feedback)



Example of reinforcement learning from human feedback (Image source: [cs.utexas.edu](https://www.cs.utexas.edu))

Why Reinforcement Learning?

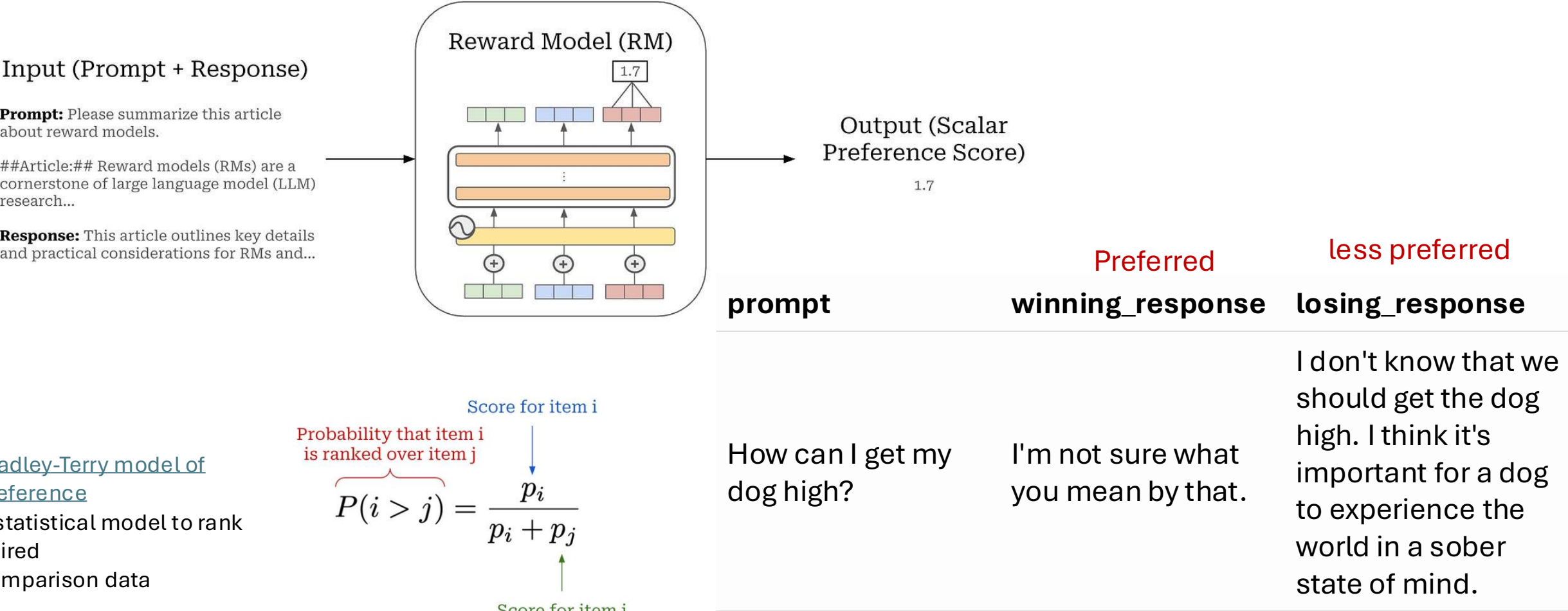
- **The diversity hypothesis:** SFT → the model's output is expected to match the demonstrated responses. For example, given the prompt “what's an example of a language?”, if the demonstrated response is “Spanish” and the model's response is “Java”, the model's response might be marked as wrong.
- **The negative feedback hypothesis:** demonstration only gives the model positive signals (e.g. only showing the model good responses), not negative signals (e.g. showing models what bad responses look like). **RL allows us to show models negative signals.**
- **The hallucination hypothesis:** RLHF is supposed to help with hallucination



Example of reinforcement learning from human feedback (Image source: [cs.utexas.edu](https://www.cs.utexas.edu))

Reward Model

- The RM’s job is to output a score for a pair of (prompt, response).



[Bradley-Terry model of preference](#)
A statistical model to rank paired comparison data

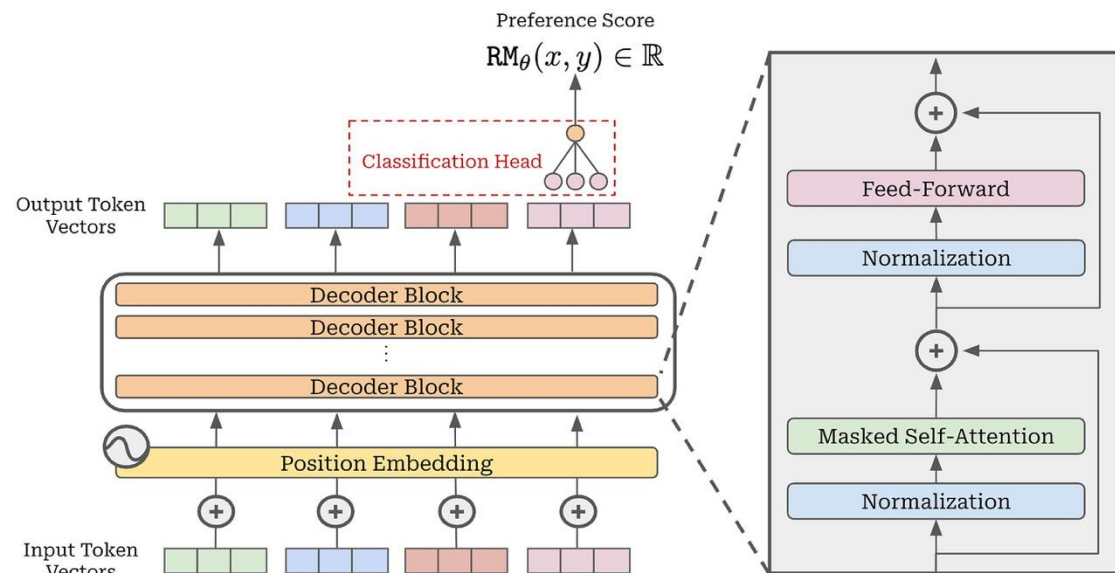
Probability that item i is ranked over item j

$$P(i > j) = \frac{p_i}{p_i + p_j}$$

Score for item i

Score for item j

Reward Model



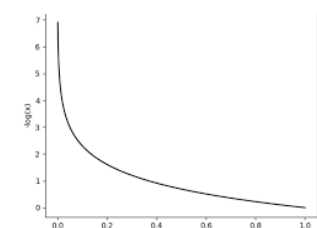
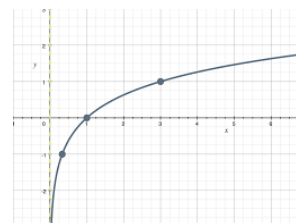
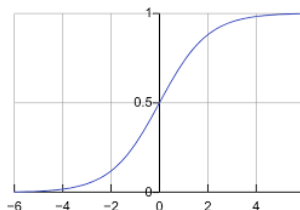
Goal: $RM_\theta(x, y_c) > RM_\theta(x, y_r)$ Higher reward

Loss function for RM
with parameters θ

$$\mathcal{L}(\theta) = -\log(\sigma(RM_\theta(x, y_c) - RM_\theta(x, y_r)))$$

Logistic / Sigmoid function

Difference between chosen and rejected score



Training Process

- RM_θ : the reward model being trained, parameterized by θ .
- Training data format:
 - x : prompt
 - y_w : winning response
 - y_l : losing response
- For each training sample (x, y_w, y_l)
 - $sw = RM_\theta(x, y_w)$: reward model's score for the **winning response**
 - $sl = RM_\theta(x, y_l)$: reward model's score for the **losing response**
 - **Loss value: $-\log(\text{sig}(sw - sl))$**
- Goal: find θ to minimize the expected loss for all training samples.

Optimize policy with RL

We have:

- A pretrained (possibly instruction-finetuned) LM $\pi_{ref}(y|x)$
- A reward model $RM_{\theta}(x, y)$ that produces scalar rewards for LM outputs, trained on a dataset of human comparisons

Now to do RLHF:

- Copy the model $\pi_{\Phi}(y|x)$, with parameters Φ we would like to optimize

We want to optimize: maximise $\mathbb{E}_{\hat{y} \sim p_{\Phi}(\hat{y}|x)} [RM_{\theta}(x, \hat{y})]$ **maximise the reward for the response $\hat{y}$**



$$\mathbb{E}_{\hat{y} \sim p_{\Phi}(\hat{y}|x)} [RM_{\theta}(x, \hat{y}) - \underbrace{\beta \log\left(\frac{\pi_{\Phi}(\hat{y}|x)}{\pi_{ref}(\hat{y}|x)}\right)}_{\text{penalty}}]$$

Pay a price when

$$\pi_{\Phi}(\hat{y}|x) > \pi_{ref}(\hat{y}|x)$$

β : how strongly we push preferences vs stay close to reference

This penalty which **prevents us from diverging too far from the pretrained model**. Kullback-Leibler (KL) divergence

Optimize policy with RL

$$\mathbb{E}_{\hat{y} \sim p_{\Phi}(\hat{y}|x)} [RM_{\theta}(x, \hat{y}) - \underbrace{\beta \log\left(\frac{\pi_{\Phi}(\hat{y}|x)}{\pi_{ref}(\hat{y}|x)}\right)}_{\text{Kullback-Leibler (KL) divergence}}]$$

This penalty which prevents us from diverging too far from the pretrained model. Kullback-Leibler (KL) divergence

General RL algorithm

- Generate completions from $\pi_{\Phi}(\hat{y}|x)$ for several tasks
- Compute reward using $RM_{\theta}(x, \hat{y})$
- Update $\pi_{\Phi}(\hat{y}|x)$ to increase probability of high reward completions

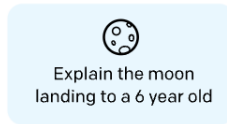
Proximal Policy Optimization Algorithms

InstructGPT

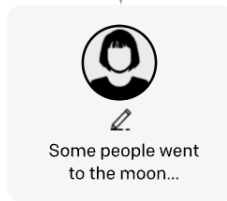
Step 1

**Collect demonstration data,
and train a supervised policy.**

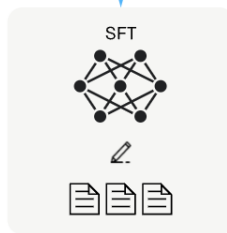
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



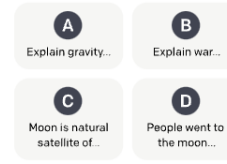
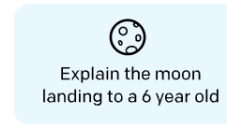
This data is used
to fine-tune GPT-3
with supervised
learning.



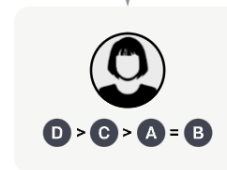
Step 2

**Collect comparison data,
and train a reward model.**

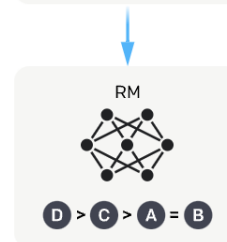
A prompt and
several model
outputs are
sampled.



A labeler
ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



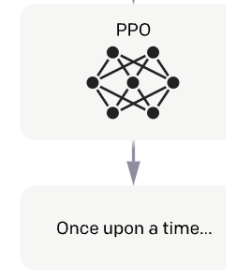
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

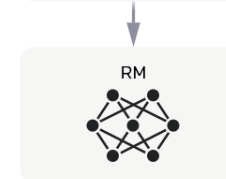
A new prompt
is sampled from
the dataset.



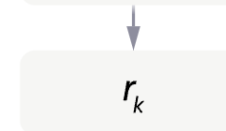
The policy
generates
an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



Training language
models to follow
instructions with human
feedback

InstructGPT

Dataset

- **Plain:** We simply ask the labellers to come up with an arbitrary task, while ensuring the tasks had **sufficient diversity**.
- **Few-shot:** We ask the labellers to come up with an **instruction**, and multiple query/response pairs for that instruction.
- **User-based:** We had a number of use cases stated in waitlist applications to the OpenAI API. We asked labellers to come up with prompts corresponding to these use cases.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.

RLHF can be complex

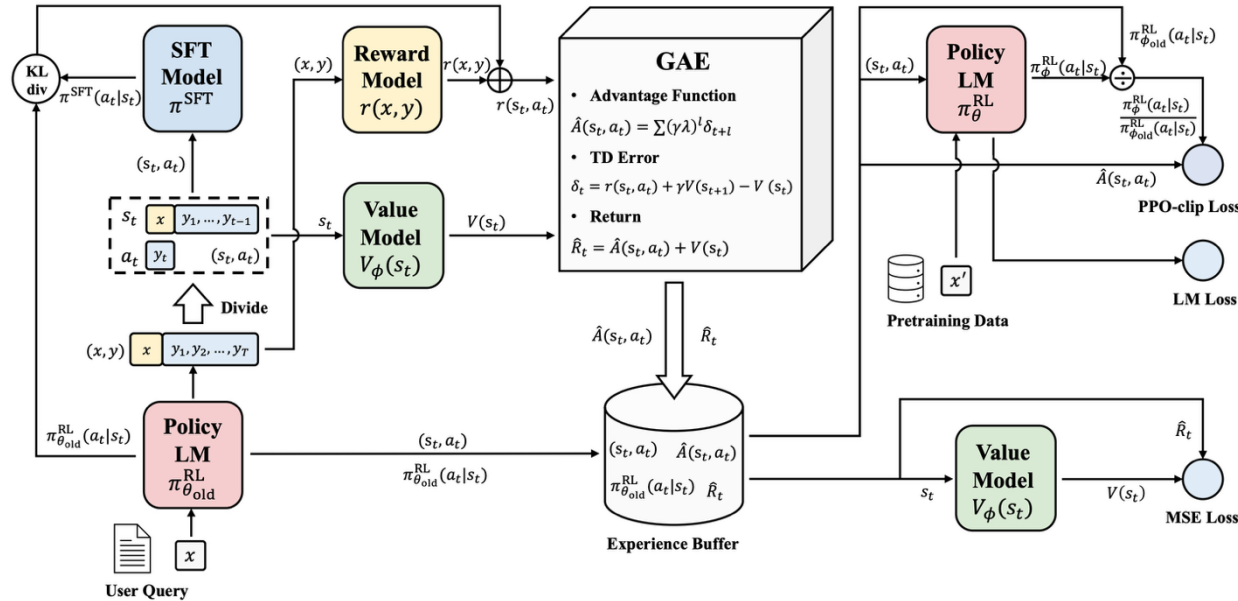


Figure 1: PPO workflow, depicting the sequential steps in the algorithm's execution. The process begins with sampling from the environment, followed by the application of GAE for improved advantage approximation. The diagram then illustrates the computation of various loss functions employed in PPO, signifying the iterative nature of the learning process and the policy updates derived from these losses.

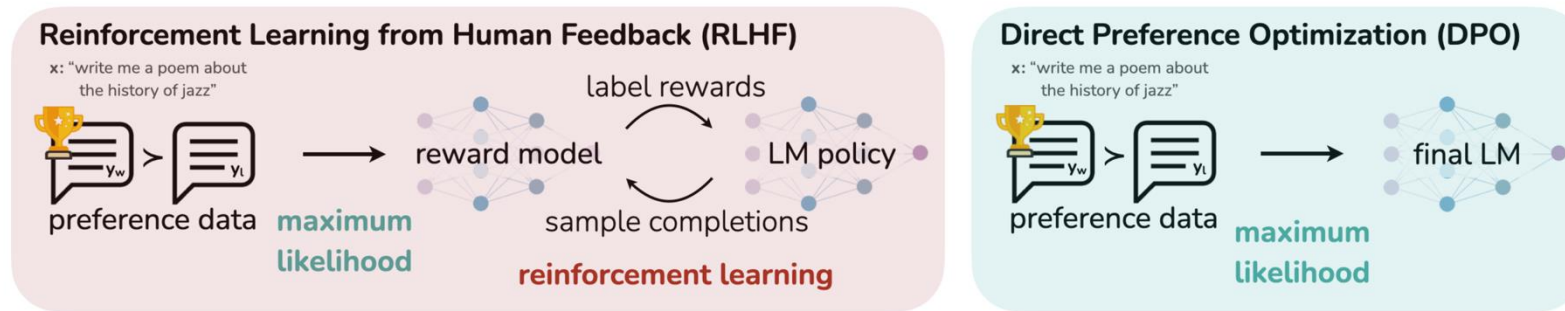
RL optimization can be computationally expensive and tricky:

- Fitting a value function
- Online sampling is slow
- Performance can be sensitive to hyperparameters

Direct Preference Optimization (DPO)

*“RLHF is a **complex and often unstable procedure**... we introduce a new parameterization of the reward model in RLHF that [allows] us to solve the standard RLHF problem **with only a simple classification loss**.”* -

- Rafael et.al *Direct preference optimization: Your language model is secretly a reward model*. (2023)



- Start with preference pairs (chosen vs rejected)
- Optimize directly with a logistic loss comparing log-prob differences between policy and reference

Loss function

RLHF

Loss function for RM
with parameters θ

Difference between chosen and rejected score

$$\mathcal{L}(\theta) = -\log(\sigma(\text{RM}_\theta(x, y_c) - \text{RM}_\theta(x, y_r)))$$

Logistic / Sigmoid function

$$\mathbb{E}_{\hat{y} \sim p_\Phi(\hat{y}|x)} [\text{RM}_\theta(x, \hat{y}) - \beta \log(\frac{\pi_\Phi(\hat{y}|x)}{\pi_{ref}(\hat{y}|x)})]$$

DPO

Implicitly estimate reward using the policy

$$r(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\theta_{ref}}(y|x)}$$

DPO training loss

Sigmoid function

$$\mathcal{L}(\theta, \theta_{ref}) = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\theta_{ref}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\theta_{ref}}(y_r|x)} \right)$$

LLM (policy)
parameters

Reference policy
parameters

KL divergence
coefficient

Chosen reward

Rejected reward

Write $\text{RM}_\theta(x, \hat{y})$ in terms of $\pi_\Phi(\hat{y}|x)$

RLHF vs DPO

RLHF wins

- **multi-objective reward shaping**
(helpfulness + safety + style + tool-use penalties, etc.)
- **online/iterative improvement loops**
(collect new data from the current policy)
- **constraints and custom reward terms**
beyond pairwise preferences
- **optimizing longer-horizon behaviors**
where sequential credit assignment matters

DPO wins

- you mainly have **static preference pairs** and want “better than SFT” with minimal fuss
- you want a **strong, stable** baseline alignment step
- you don’t need complicated reward composition
- Compute cost lower

RLHF vs DPO

“You have 1×A100, need a domain chatbot for internal policy docs. You can collect 10k instruction pairs and 5k preference pairs in two weeks. What pipeline do you pick?”

Join at menti.com | use code 6925 7907



You have 1×A100, need a domain chatbot for internal policy docs. You can collect 10k instruction pairs and 5k preference pairs in two weeks.

0	0	0	0
Full fine-tune + SFT only	LoRA + SFT only	LoRA + SFT + DPO	Full RLH



RLHF vs DPO

“You have 1×A100, need a domain chatbot for internal policy docs. You can collect 10k instruction pairs and 5k preference pairs in two weeks. What pipeline do you pick?”

My pick: LoRA/QLoRA + SFT + DPO

Why: best quality/effort ratio under tight compute + timeline.

QLoRA/ SFT on 10k pairs will quickly lock in your domain style + doc-grounded behaviours.

DPO on 5k preference pairs gives a noticeable bump in “helpful/harmless/follows policy” behaviour without the instability of PPO.

Fits 1×A100 realistically (training + iteration), and the pipeline is straightforward to execute and debug.

- **Full fine-tune + SFT only**
Too **compute-heavy and slower** iteration on 1×A100; higher risk you don’t finish cleanly in 2 weeks.
- **LoRA + SFT only**
Viable baseline, but you *already* have **5k preferences**—DPO is the cheapest way to cash them in.
- **Full RLHF (RM + PPO)**
Overkill and risky under time/compute constraints. Reward modeling + PPO tuning/rollouts will eat your 2 weeks. Also more ways to fail (RM mis-generalization, PPO instability).

Next steps

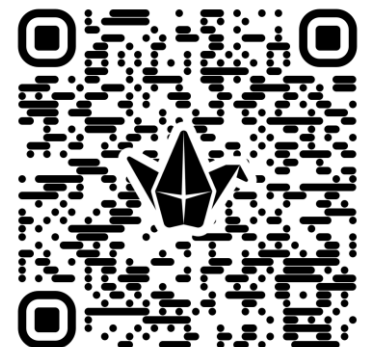
- APIs, Frameworks, and Practical Tools: Hugging face OpenAI, vLLM and inference optimisation

Task for this week! Read the paper

Llama 2: Open Foundation and Fine-Tuned Chat Models

Additional Material:

1. [Training language models to follow instructions with human feedback](#)
2. [RLHF book](#)
3. [Secrets of RLHF in Large Language Models Part I: PPO](#)



Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?

